

Guida Teorica — Progetto 25

Extreme Learning con regolarizzazione L_1 :
Iteratively Reweighted Least Squares e Deflected Subgradient
Computational Mathematics for Learning and Data Analysis (646AA)

A.A. 2025/26

Indice

1	Il problema: Extreme Learning Machine con regolarizzazione L_1	3
1.1	Che cos'è un'Extreme Learning Machine?	3
1.2	Perché la regolarizzazione L_1 ?	4
1.3	Il problema che dobbiamo risolvere	4
1.4	Analisi della funzione obiettivo	4
1.5	Quando abbiamo trovato la soluzione?	5
2	Richiami di Algebra Lineare Numerica	6
2.1	Minimi quadrati lineari: trovare la “retta migliore”	6
2.2	Fattorizzazione di Cholesky: risolvere il sistema in modo furbo	7
2.3	Gradiente Coniugato: un'alternativa iterativa	7
2.4	Condizionamento: quanto è “difficile” il nostro sistema?	8
3	Algoritmo A1: Iteratively Reweighted Least Squares (IRLS)	9
3.1	L'idea: trasformare un problema difficile in tanti problemi facili	9
3.2	Il sotto-problema: minimi quadrati pesati	9
3.3	Algoritmo completo	10
3.4	Perché converge? E quanto velocemente?	10
4	Richiami di Ottimizzazione Non-Smooth	11
4.1	Subgradienti: il gradiente per funzioni con spigoli	11
4.2	Perché l'ottimizzazione non-smooth è più difficile	12
4.3	Metodi del subgradiente: la relazione fondamentale	13
5	Algoritmo A2: Deflected Subgradient Method	14
5.1	Il problema del subgradiente puro e l'idea della deflection	14
5.2	Come scegliere la deflection e il passo	14
5.3	L'algoritmo completo con target level	15
5.4	Calcolo pratico del subgradiente per il nostro problema	16
5.5	Convergenza: cosa aspettarsi	17
6	Proprietà attese dalla teoria	17
6.1	Per l'algoritmo A1 (IRLS)	17
6.2	Per l'algoritmo A2 (Deflected Subgradient)	18
6.3	Chi “vince”? Confronto A1 vs A2	18

7	Considerazioni aggiuntive	18
7.1	Scelta della funzione di attivazione	18
7.2	Confronto con solutori di riferimento	19
7.3	Smoothness e strong convexity del nostro problema	19
8	Mappa delle lezioni: cosa studiare e dove trovarlo	20
8.1	Lezioni di Ottimizzazione	20
8.2	Lezioni di Algebra Lineare Numerica	22
8.3	Tabella riassuntiva: priorità di studio	25

1 Il problema: Extreme Learning Machine con regolarizzazione L_1

1.1 Che cos'è un'Extreme Learning Machine?

Intuizione

Immagina di voler insegnare a un computer a prevedere qualcosa — ad esempio, il prezzo di una casa dato il numero di stanze, i metri quadri, la zona, ecc. Un modo per farlo è costruire una “macchina” che prende questi numeri in ingresso, li trasforma in modo complicato, e poi combina i risultati con dei “pesi” per ottenere la previsione.

Una rete neurale classica impara *tutti* i pesi — sia quelli della trasformazione iniziale, sia quelli della combinazione finale. L'**Extreme Learning Machine** (ELM) fa una scelta drastica: i pesi della trasformazione iniziale vengono **tirati a caso e mai cambiati**. Si imparano solo i pesi finali. Sembra una follia, ma funziona sorprendentemente bene — e soprattutto, trasforma un problema difficilissimo (imparare una rete neurale) in un problema molto più semplice che sappiamo risolvere bene: un sistema di equazioni lineari.

Più formalmente, un'ELM è una rete neurale a singolo strato nascosto dove:

1. Si genera una matrice di pesi casuali $W_1 \in \mathbb{R}^{p \times d}$ (con p neuroni nascosti e d input) e la si **fissa per sempre**.
2. Per ogni input x , si calcolano le “feature nascoste” applicando una funzione di attivazione σ (sigmoide, ReLU, tanh...): $\sigma(W_1 x) \in \mathbb{R}^p$.
3. La previsione è una combinazione lineare di queste feature: $\hat{y} = w^T \sigma(W_1 x)$, dove $w \in \mathbb{R}^n$ sono i **pesi da imparare**.

Per un dataset di m campioni, possiamo raccogliere tutte le feature nascoste in una matrice:

$$X = \begin{bmatrix} \sigma(W_1 x^{(1)})^T \\ \vdots \\ \sigma(W_1 x^{(m)})^T \end{bmatrix} \in \mathbb{R}^{m \times n}, \quad y \in \mathbb{R}^m.$$

Analogia

Pensa a X come a una tabella: ogni riga è un campione, ogni colonna è una “feature trasformata”. I pesi w dicono “quanto conta” ciascuna colonna per la previsione. Trovare i pesi migliori significa risolvere un problema di minimi quadrati: “trova i pesi tali che Xw sia il più vicino possibile a y ”.

1.2 Perché la regolarizzazione L_1 ?

Intuizione

Immagina di avere 1000 colonne nella tua tabella X (cioè 1000 feature). Probabilmente molte sono irrilevanti o ridondanti. Se cerchi i pesi che minimizzano l'errore *senza nessun vincolo*, il modello potrebbe “impazzire” e assegnare pesi enormi alle feature irrilevanti per compensare il rumore nei dati — come uno studente che impara a memoria le risposte del compito senza capire la materia. Questo fenomeno si chiama **overfitting**: il modello funziona benissimo sui dati di allenamento ma malissimo su dati nuovi.

La **regolarizzazione** aggiunge una “penalità” ai pesi troppo grandi: “ok, cerca di prevedere bene, ma senza esagerare con i pesi”. La regolarizzazione L_1 (chiamata LASSO) è speciale perché non solo tiene i pesi piccoli, ma ne porta molti esattamente a **zero**. In pratica, il modello “sceglie” automaticamente di usare solo poche feature e ignorare le altre. Questo si chiama **feature selection** ed è estremamente utile: il modello risultante è più semplice, più interpretabile e generalizza meglio [8, cf. §5.1.2].

Perché proprio la norma L_1 e non la L_2 ? Esistono diverse possibilità per la penalità:

- **Regolarizzazione L_2 (Ridge):** penalizza $\sum_i w_i^2$. Rende i pesi piccoli, ma *non* li porta a zero. È liscia (differenziabile ovunque), quindi facile da ottimizzare.
- **Regolarizzazione L_0 :** penalizza il *numero* di pesi non nulli. L'ideale per la sparsità, ma è una funzione orribile — non è nemmeno continua, e ottimizzarla è NP-hard.
- **Regolarizzazione L_1 (LASSO):** penalizza $\sum_i |w_i|$. È la **migliore approssimazione convessa** della norma L_0 [8, Appendice A.8]. Promuove sparsità come la L_0 , ma è convessa come la L_2 . Il prezzo da pagare è che non è differenziabile (ha degli “spigoli” dove $w_i = 0$).

Analogia

Pensa alle palle unitarie di queste norme in 2D: la palla L_2 è un cerchio, la palla L_1 è un rombo (quadrato ruotato), la palla L_0 ... non è nemmeno un insieme convesso. Il rombo L_1 ha dei *vertici* sugli assi: quando le curve di livello della funzione di errore “toccano” il rombo, lo fanno spesso in corrispondenza di un vertice, cioè in un punto dove una componente di w è esattamente zero. Ecco perché la L_1 promuove la sparsità!

1.3 Il problema che dobbiamo risolvere

Il nostro obiettivo è trovare w che minimizzi:

$$\min_{w \in \mathbb{R}^n} f(w) = \underbrace{\|Xw - y\|_2^2}_{\text{errore di previsione}} + \underbrace{\lambda \|w\|_1}_{\text{penalità di sparsità}} \quad (1)$$

dove $\lambda > 0$ è un parametro scelto dall'utente che bilancia i due obiettivi: λ piccolo $\Rightarrow$ si privilegia la precisione; λ grande $\Rightarrow$ si privilegia la semplicità (sparsità).

In poche parole

Il problema (1) chiede: “trova i pesi w che prevedano bene (errore piccolo) usando il minor numero possibile di feature (pesi sparsi)”. È un problema LASSO classico.

1.4 Analisi della funzione obiettivo

La funzione f è la somma di due pezzi con proprietà molto diverse:

Pezzo 1: l'errore quadratico $g(w) = \|Xw - y\|_2^2$

Intuizione

Questa è la parte “facile”. Misura quanto le previsioni Xw sono lontane dai valori veri y . È una funzione **liscia** (differenziabile quante volte vuoi) e **convessa** (ha la forma di una “ciotola” — non ha minimi locali fasulli). Se X ha rango pieno per colonne (cioè non ha colonne “ridondanti”), la ciotola ha un unico punto più basso.

Espandendo il quadrato: $g(w) = w^T X^T X w - 2y^T X w + y^T y$.

Il gradiente (la “freccia” che indica la direzione in cui g cresce più velocemente) è:

$$\nabla g(w) = 2X^T(Xw - y) = 2X^T X w - 2X^T y.$$

L'Hessiano (che descrive la “curvatura” della ciotola) è $\nabla^2 g(w) = 2X^T X \succeq 0$: è costante e semi-definito positivo, il che conferma che g è convessa.

Pezzo 2: la penalità L_1 , $h(w) = \lambda \|w\|_1 = \lambda \sum_{i=1}^n |w_i|$

Intuizione

Questa è la parte “difficile”. La funzione valore assoluto $|w_i|$ è convessa, ma ha uno **spigolo** in $w_i = 0$: se disegni il grafico di $|x|$ vedi una “V”. In quello spigolo la derivata non esiste — il grafico “gira a gomito” e la pendenza a sinistra (-1) è diversa da quella a destra ($+1$).

Questo spigolo è *cruciale* per la sparsità: è come una “buca” che “cattura” i pesi e li porta a zero. Ma è anche ciò che rende il problema matematicamente più complicato, perché non possiamo usare i metodi classici che richiedono il gradiente.

Proposizione 1.1 (Convessità di f). *La funzione $f = g + h$ è convessa (somma di funzioni convesse). Se X ha rango pieno per colonne, f è strettamente convessa e ammette un unico minimo globale.*

Dimostrazione. g è convessa poiché $\nabla^2 g = 2X^T X \succeq 0$. h è convessa in quanto norma. La somma di funzioni convesse è convessa. Se $\text{rank}(X) = n$ allora $X^T X \succ 0$, dunque g è strettamente convessa; la somma con la convessa h resta strettamente convessa, garantendo unicità del minimo [5, slide 3, set 3]. $\square$

Osservazione 1.2 (Non-differenziabilità). f non è differenziabile ovunque. Nei punti dove $w_i \neq 0 \forall i$, il gradiente esiste ed è $\nabla f(w) = 2X^T(Xw - y) + \lambda \text{sign}(w)$, dove $\text{sign}(w)_i = +1$ se $w_i > 0$, -1 se $w_i < 0$. Quando qualche $w_i = 0$, la derivata parziale non esiste e si deve ragionare in termini di *subgradienti* (vedi Sezione 4).

1.5 Quando abbiamo trovato la soluzione?

Intuizione

Per una funzione liscia, sappiamo di essere al minimo quando il gradiente è zero: la “freccia” che indica la direzione di crescita scompare. Per una funzione con spigoli come la nostra, il gradiente non esiste negli spigoli. Serve uno strumento più potente: il **subdifferenziale**, che è un *insieme* di “quasi-gradienti” (vedi Sezione 4.1).

La regola è: siamo al minimo di f se e solo se il vettore zero “sta dentro” il subdifferenziale, cioè $0 \in \partial f(w^*)$.

Per il nostro problema:

$$\partial f(w) = 2X^T(Xw - y) + \lambda \partial \|w\|_1,$$

dove il subdifferenziale della norma L_1 è:

$$[\partial \|w\|_1]_i = \begin{cases} \{+1\} & \text{se } w_i > 0, \\ \{-1\} & \text{se } w_i < 0, \\ [-1, +1] & \text{se } w_i = 0. \end{cases}$$

La condizione $0 \in \partial f(w^*)$ si può scrivere componente per componente: per ogni $i = 1, \dots, n$,

$$[2X^T(Xw^* - y)]_i + \lambda s_i = 0, \quad s_i \in \partial |w_i^*|. \quad (2)$$

Intuizione

Cosa dice questa condizione in pratica?

- Se alla soluzione $w_i^* \neq 0$: la derivata della parte liscia, più $\pm\lambda$, deve fare zero. È come la condizione classica “gradiente = 0” con un termine costante in più.
- Se alla soluzione $w_i^* = 0$: la derivata della parte liscia, in valore assoluto, deve essere **al massimo** λ . Se fosse più grande di λ , vorrebbe dire che c’è una “spinta” abbastanza forte da tirare w_i via da zero; ma se è più piccola, la penalità L_1 è abbastanza forte da “tenere” w_i bloccato a zero.

Ecco perché λ controlla la sparsità: più λ è grande, più è facile che la condizione $|\text{derivata}|_i \leq \lambda$ sia soddisfatta, e quindi più componenti restano a zero.

2 Richiami di Algebra Lineare Numerica

Intuizione

Questa sezione serve perché l’algoritmo IRLS (Sezione 3) ad ogni iterazione deve risolvere un **sistema di equazioni lineari**. Per capire l’IRLS serve quindi capire come si risolvono questi sistemi in modo efficiente e stabile. I concetti chiave sono tre: le *equazioni normali* (la formulazione del problema), la *fattorizzazione di Cholesky* (il metodo diretto per risolverlo) e il *gradiente coniugato* (il metodo iterativo).

2.1 Minimi quadrati lineari: trovare la “retta migliore”

Analogia

Hai un foglio con dei punti sparsi e vuoi tracciare la retta che ci passa “più vicino”. Matematicamente, stai cercando il vettore w tale che Xw (le previsioni) sia il più vicino possibile a y (i dati veri). “Più vicino” nel senso dei minimi quadrati: minimizzare la somma dei quadrati degli errori.

Il problema è $\min_{w \in \mathbb{R}^n} \|Xw - y\|_2^2$. La soluzione si trova imponendo che il gradiente sia zero:

$$2X^T(Xw - y) = 0 \implies X^T Xw = X^T y.$$

Teorema 2.1 (Equazioni normali [7, Lecture 4]). Se $X \in \mathbb{R}^{m \times n}$ ha rango pieno per colonne ($m \geq n$, $\text{rank}(X) = n$), allora la soluzione unica è:

$$X^T Xw^* = X^T y \implies w^* = (X^T X)^{-1} X^T y.$$

La matrice $X^T X \in \mathbb{R}^{n \times n}$ è simmetrica e definita positiva.

Intuizione

Perché “equazioni *normali*”? Il nome viene dal fatto che l'errore $Xw^* - y$ risulta *ortogonale* (= “normale”) allo spazio delle colonne di X . In parole povere: l'errore residuo non può essere “spiegato” da nessuna combinazione delle colonne di X — abbiamo estratto tutta l'informazione possibile.

2.2 Fattorizzazione di Cholesky: risolvere il sistema in modo furbo

Analogia

Risolvere il sistema $Qw = b$ (dove $Q = X^T X$ è una matrice simmetrica definita positiva) è come smontare un cubo di Rubik: si può fare brutalmente, oppure si può sfruttare la *struttura* del problema. La fattorizzazione di Cholesky sfrutta il fatto che Q è simmetrica e positiva per “spezzarla” in un prodotto di due matrici triangolari: $Q = LL^T$.

Risolvere $LL^T w = b$ è molto più facile: prima risolvi $Lv = b$ (andando dall'alto verso il basso, visto che L è triangolare inferiore), poi $L^T w = v$ (dal basso verso l'alto). Due passi facili invece di uno difficile.

Teorema 2.2 (Cholesky [7, Lecture 20], [8, §1.6]). *Se $Q \in \mathbb{R}^{n \times n}$ è simmetrica definita positiva, allora esiste un'unica matrice triangolare inferiore L con diagonale positiva tale che $Q = LL^T$. Il costo computazionale è circa $n^3/3$ operazioni — la metà di una fattorizzazione LU generica, perché sfruttiamo la simmetria.*

Il sistema $Qw = b$ si risolve poi in due passi, ciascuno dal costo $O(n^2)$:

1. *Forward substitution*: si risolve $Lv = b$ partendo dalla prima equazione;
2. *Back substitution*: si risolve $L^T w = v$ partendo dall'ultima equazione.

In poche parole

Per risolvere il sistema delle equazioni normali $X^T X w = X^T y$: (1) calcola $Q = X^T X$ (costo $O(mn^2)$, da fare **una sola volta**); (2) fattorizza $Q = LL^T$ con Cholesky (costo $O(n^3/3)$); (3) risolvi con forward/back substitution (costo $O(n^2)$). Il passo (2) va rifatto ad ogni iterazione IRLS, perché la matrice Q cambia.

2.3 Gradiente Coniugato: un'alternativa iterativa

Intuizione

E se la matrice fosse troppo grande per la fattorizzazione di Cholesky? Ad esempio, con $n = 100\,000$ variabili, $n^3/3 \approx 3 \times 10^{14}$ operazioni sono troppe. In questi casi si può usare un metodo *iterativo*: non si calcola la soluzione esatta, ma una sequenza di approssimazioni sempre migliori.

Il **Gradiente Coniugato** (CG) è il metodo iterativo più elegante per sistemi con matrice simmetrica definita positiva. L'idea è: ad ogni passo, cerca la soluzione nella direzione “migliore” che non sia già stata esplorata. In teoria converge in al più n passi (come un metodo diretto!), ma in pratica spesso bastano molti meno passi per avere una buona approssimazione.

Definizione 2.3 (Direzioni Q -coniugate [7, Lecture 5]). I vettori $d_0, d_1, \dots$ sono detti Q -coniugati se $d_i^T Q d_j = 0$ per $i \neq j$.

Analogia

Pensa a un terreno collinare con una forma ellittica. Le “direzioni coniugate” sono come le direzioni “indipendenti” lungo cui la ciotola si allunga: una volta che hai minimizzato lungo una di queste direzioni, non devi più tornare indietro — il lavoro fatto resta valido. In contrasto, il gradiente ordinario “zig-zaga” avanti e indietro, sprecando passi. Il CG elimina questo zig-zag.

Teorema 2.4 (Convergenza di CG [7, Lecture 5], [8, §1.5]). *Il metodo CG con $Q \succ 0$ converge in al più n iterazioni (in aritmetica esatta). La velocità di convergenza dipende dal numero di condizione $\kappa(Q) = \lambda_{\max}(Q)/\lambda_{\min}(Q)$:*

$$\frac{\|w_k - w^*\|_Q}{\|w_0 - w^*\|_Q} \leq 2 \left(\frac{\sqrt{\kappa(Q)} - 1}{\sqrt{\kappa(Q)} + 1} \right)^k.$$

Intuizione

Cosa dice questo bound? Se il numero di condizione κ è vicino a 1 (cioè la “ciotola” è quasi sferica), il CG converge in pochissimi passi. Se κ è enorme (la ciotola è molto “schiacciata”, come un ellissoide sottile), il CG fa fatica e potrebbe servire quasi tutto n passi. Il numero di condizione è il “nemico” di tutti i metodi iterativi.

2.4 Condizionamento: quanto è “difficile” il nostro sistema?

Definizione 2.5 (Numero di condizione [7, Lecture 12]). Per una matrice A non singolare, il numero di condizione è $\kappa(A) = \|A\|_2 \|A^{-1}\|_2 = \sigma_{\max}(A)/\sigma_{\min}(A)$: il rapporto tra il valore singolare più grande e il più piccolo.

Analogia

Il numero di condizione misura quanto il sistema è “sensibile ai disturbi”. Immagina di stare su una cresta stretta in montagna: un piccolo spostamento laterale e cadi nel precipizio (κ grande). Se invece sei in fondo a una valle larga e poco profonda, puoi spostarti un po’ senza problemi (κ piccolo).
Nel nostro caso, $\kappa(X^T X) = \kappa(X)^2$. Se X è già un po’ mal condizionata, passando a $X^T X$ il condizionamento *peggiora al quadrato*. Ecco perché le equazioni normali possono essere instabili, e perché alternative come la fattorizzazione QR o SVD sono a volte preferibili [7, Lectures 10–11].

Attenzione

Per il nostro progetto, il condizionamento di $X^T X$ dipende dalla matrice delle feature dell’ELM, che a sua volta dipende dai pesi casuali W_1 e dalla funzione di attivazione. Se il condizionamento è cattivo, l’IRLS potrebbe convergere lentamente o dare risultati numericamente imprecisi.

3 Algoritmo A1: Iteratively Reweighted Least Squares (IRLS)

3.1 L'idea: trasformare un problema difficile in tanti problemi facili

Intuizione

Il nostro problema è difficile perché la norma L_1 ha degli spigoli. La parte quadratica $\|Xw - y\|^2$ da sola sarebbe facilissima — basterebbe risolvere un sistema di equazioni lineari. Ma la norma $\|w\|_1$ rovina tutto.

L'idea geniale dell'IRLS è: **ad ogni iterazione, approssima la norma L_1 con una funzione quadratica** (cioè “liscia”, senza spigoli) costruita usando la soluzione corrente. In questo modo, ad ogni passo si deve risolvere solo un problema di minimi quadrati — cosa che sappiamo fare benissimo con Cholesky o CG.

Ovviamente l'approssimazione è imperfetta, quindi si ripete: con la nuova soluzione si costruisce un'approssimazione migliore, si risolve, e così via. Da qui il nome: “Iteratively Reweighted” — i pesi (weights) cambiano ad ogni iterazione.

Come si approssima $|w_i|$ con un quadratico? L'osservazione chiave è quasi banale. Se conosco il valore corrente $w_{k,i} \neq 0$, posso scrivere:

$$|w_i| = \frac{w_i^2}{|w_i|} \approx \frac{w_i^2}{|w_{k,i}|}.$$

Questa è un'approssimazione di $|w_i|$ che è *quadratica* in w_i (il denominatore $|w_{k,i}|$ è un numero fisso, non una variabile). Dunque:

$$\|w\|_1 = \sum_i |w_i| \approx \sum_i \frac{w_i^2}{|w_{k,i}|} = w^T W_k w,$$

dove W_k è una matrice diagonale con $(W_k)_{ii} = |w_{k,i}|^{-1/2}$.

Analogia

È come approssimare una V (il grafico di $|x|$) con una parabola che la “tocca” nel punto corrente. La parabola non è perfetta — in particolare, vicino a zero, $|x|$ ha lo spigolo mentre la parabola è liscia — ma è una buona approssimazione locale. Iterando, la parabola si “adatta” alla V sempre meglio.

Attenzione

Problema: cosa succede se $w_{k,i} \approx 0$? Il peso $|w_{k,i}|^{-1/2}$ esplode! Per evitare divisioni per (quasi) zero, si introduce un **threshold di sicurezza** $\varepsilon_{\text{thr}} > 0$:

$$(W_k)_{ii} = \max(|w_{k,i}|, \varepsilon_{\text{thr}})^{-1/2}.$$

Valori tipici: $\varepsilon_{\text{thr}} \in [10^{-6}, 10^{-10}]$. È un parametro algoritmico cruciale: troppo grande e l'approssimazione è grossolana; troppo piccolo e si rischiano instabilità numeriche.

3.2 Il sotto-problema: minimi quadrati pesati

Ad ogni iterazione k , sostituendo l'approssimazione quadratica, si risolve:

$$w_{k+1} = \arg \min_w \|Xw - y\|_2^2 + 2\lambda \|W_k w\|_2^2. \quad (3)$$

Intuizione

Questo è un problema di minimi quadrati con un termine di regolarizzazione L_2 (Ridge) “pesato”. La matrice W_k gioca il ruolo di “pesi personalizzati” per ogni componente: le componenti di w_k che sono piccole ricevono pesi grandi (forte penalizzazione), incoraggiandole ad andare a zero; le componenti grandi ricevono pesi piccoli (debole penalizzazione), lasciandole libere.

Proposizione 3.1 (Equazioni normali pesate). *La soluzione di (3) soddisfa:*

$$(X^T X + 2\lambda W_k^T W_k) w_{k+1} = X^T y. \quad (4)$$

Dimostrazione. Il gradiente del sotto-problema è $\nabla f_k(w) = 2X^T(Xw - y) + 4\lambda W_k^T W_k w$. Ponendo a zero e dividendo per 2: $(X^T X + 2\lambda W_k^T W_k)w = X^T y$. La matrice è simmetrica definita positiva (somma di $X^T X \succeq 0$ e $2\lambda W_k^T W_k \succ 0$ se $\varepsilon_{thr} > 0$), dunque la soluzione è unica. $\square$

In poche parole

Ogni iterazione IRLS = risolvere un sistema lineare $n \times n$ con matrice SPD. Si può usare Cholesky ($O(n^3/3)$) o CG. Il trucco: $X^T X$ si pre-calcola **una sola volta**. Ad ogni iterazione cambia solo la diagonale $2\lambda W_k^T W_k$ — un’operazione $O(n)$.

3.3 Algoritmo completo

Algorithm 1 Iteratively Reweighted Least Squares (IRLS) per LASSO

Require: $X \in \mathbb{R}^{m \times n}$, $y \in \mathbb{R}^m$, $\lambda > 0$, $\varepsilon_{thr} > 0$, $\varepsilon_{stop} > 0$, k_{max}

```

1: Pre-calcola  $A = X^T X$  e  $b = X^T y$ 
2: Inizializza  $w_0$  (es. soluzione minimi quadrati:  $w_0 = A^{-1}b$ )
3: for  $k = 0, 1, 2, \dots, k_{max}$  do
4:   Calcola  $W_k = \text{diag}(\max(|w_{k,1}|, \varepsilon_{thr})^{-1/2}, \dots, \max(|w_{k,n}|, \varepsilon_{thr})^{-1/2})$ 
5:   Risolvi  $(A + 2\lambda W_k^T W_k) w_{k+1} = b$  (Cholesky o CG)
6:   if  $\|w_{k+1} - w_k\| / \max(1, \|w_k\|) < \varepsilon_{stop}$  then
7:     break (convergenza raggiunta)
8:   end if
9: end for
10: return  $w_{k+1}$ 

```

3.4 Perché converge? E quanto velocemente?

Intuizione

L’IRLS è un caso di *Majorization-Minimization* (MM): ad ogni passo costruiamo una funzione (il sotto-problema quadratico) che “sta sopra” alla funzione originale e la “tocca” nel punto corrente. Minimizzando questa funzione “che sta sopra”, si garantisce di fare almeno altrettanto bene quanto il punto corrente sul sotto-problema. Iterando, la sequenza dei valori converge.

Un punto tecnico: la convergenza è garantita rispetto ai *sotto-problemi*, non necessariamente rispetto alla funzione f originale. In pratica $f(w_k)$ potrebbe fare qualche oscillazione, ma alla fine converge.

Teorema 3.2 (Consistenza al punto fisso). *Se la sequenza $\{w_k\}$ generata da IRLS converge a un punto w^* con $|(w^*)_i| > \varepsilon_{thr}$ per ogni i , allora w^* soddisfa le condizioni di ottimalità (2) del problema originale.*

Sketch. Se $w_k = w_{k+1} = w^*$ (punto fisso), dalle equazioni normali pesate: $X^T(Xw^* - y) + 2\lambda W_*^T W_* w^* = 0$. Poiché $|w_i^*| > \varepsilon_{\text{thr}}$, il peso è $(W_*)_{ii} = |w_i^*|^{-1/2}$, e $(W_*^T W_* w^*)_i = |w_i^*|^{-1} w_i^* = \text{sign}(w_i^*)$. Sostituendo si ottiene la condizione di ottimalità. $\square$

Osservazione 3.3 (Velocità di convergenza). IRLS converge tipicamente in modo **lineare**: ad ogni iterazione l'errore si riduce approssimativamente di un fattore costante (es. l'errore si dimezza ad ogni passo). In un grafico logaritmico, questo appare come una **retta**. La velocità dipende dal condizionamento di $X^T X$ e dalla scelta di ε_{thr} .

Osservazione 3.4 (Costo per iterazione). • Aggiornamento di W_k : $O(n)$ — basta scorrere le componenti di w_k .
 • Modifica della matrice: $O(n)$ — si aggiunge la diagonale a $X^T X$ (pre-calcolata).
 • Risoluzione del sistema: $O(n^3/3)$ con Cholesky, oppure $O(kn^2)$ con CG.
 Le iterazioni sono *poche* (tipicamente 10–50) ma *costose* ($O(n^3)$ ciascuna).

4 Richiami di Ottimizzazione Non-Smooth

Intuizione

Questa sezione prepara il terreno per l'algoritmo A2 (deflected subgradient). Il problema è: la nostra funzione f non è differenziabile ovunque. I metodi classici (gradiente, Newton, CG...) richiedono tutti il gradiente. Che si fa?

La risposta è: si usa una generalizzazione del gradiente chiamata **subgradiente**. È meno preciso del gradiente, e i metodi basati su di esso sono più lenti. Ma funzionano! E per il nostro problema (dove la non-differenziabilità è “relativamente semplice” — solo degli spigoli) si comportano ragionevolmente bene.

4.1 Subgradienti: il gradiente per funzioni con spigoli

Analogia

Hai presente quando il professore dice “il gradiente è la direzione di massima crescita”? Beh, per una funzione liscia c'è *una sola* direzione di massima crescita in ogni punto. Ma per una funzione con uno spigolo (come la V di $|x|$), nello spigolo ci sono *tante* possibili “pendenze”.

Immagina di stare sulla punta della V: la pendenza a sinistra è -1 , la pendenza a destra è $+1$. Quale usi? Qualsiasi numero tra -1 e $+1$ è un “subgradiente valido”. In punti dove la funzione è liscia, c'è un solo subgradiente e coincide col gradiente.

Definizione 4.1 (Subgradiente [5, slide 5, set 5]). Sia $f : \mathbb{R}^n \rightarrow \mathbb{R}$ convessa. Un vettore $s \in \mathbb{R}^n$ è un *subgradiente* di f in x se

$$f(z) \geq f(x) + \langle s, z - x \rangle \quad \forall z \in \mathbb{R}^n.$$

L'insieme di tutti i subgradienti $\partial f(x) = \{s : s \text{ è subgradiente in } x\}$ si chiama *subdifferenziale*.

Intuizione

La definizione dice: s è un subgradiente se il piano $f(x) + \langle s, z - x \rangle$ “sta sotto” al grafico di f ovunque. Per funzioni lisce questo è il piano tangente (unico). Per funzioni con spigoli, ci sono tanti piani che stanno sotto: il subdifferenziale è l'insieme di tutte le “pendenze” di questi piani.

Proposizione 4.2 (Proprietà del subdifferenziale [5, slide 5–7, set 5]).

1. $\partial f(x)$ è un insieme convesso e compatto (chiuso e limitato).
2. Se f è differenziabile in x , allora $\partial f(x) = \{\nabla f(x)\}$ — il subdifferenziale contiene solo il gradiente.
3. x^* è minimo globale se e solo se $0 \in \partial f(x^*)$.
4. Regola della somma: $\partial(f + g)(x) = \partial f(x) + \partial g(x)$ se f e g sono convesse.

Esempio 4.3 (Subdifferenziale della norma L_1 [5, slide 7, set 5]). Per $h(w) = \|w\|_1 = \sum_i |w_i|$, il subdifferenziale è separabile (si calcola componente per componente):

$$[\partial h(w)]_i = \begin{cases} \{+1\} & w_i > 0 \text{ (pendenza della V a destra),} \\ \{-1\} & w_i < 0 \text{ (pendenza della V a sinistra),} \\ [-1, +1] & w_i = 0 \text{ (nello spigolo: qualsiasi pendenza tra } -1 \text{ e } +1). \end{cases}$$

4.2 Perché l'ottimizzazione non-smooth è più difficile

Intuizione

Il gradiente è un'informazione molto precisa: ti dice esattamente dove andare per scendere più velocemente. Il subgradiente è un'informazione più “vaga”: ti dice che “da qualche parte in questa zona si scende”, ma non necessariamente ti indica la direzione migliore. Peggio: un subgradiente potrebbe **non essere una direzione di discesa!** (Vedi l'esempio in [8, §5.1.2]: nello spigolo di $|x|$, il subgradiente $s = 0$ non punta da nessuna parte). Questa imprecisione ha conseguenze gravi sulla velocità di convergenza: i metodi basati sui subgradienti sono *intrinsecamente* più lenti di quelli basati sui gradienti.

Teorema 4.4 (Complessità dell'ottimizzazione [5, slide 8, set 5], [1]).

Regolarità	Convessità	Complessità	In pratica
Liscia, fortemente convessa	migliore	$O(\log(1/\varepsilon))$	Molto veloce
Non-smooth, fortemente convessa	peggiore	$\Omega(1/\varepsilon)$	Lento
Liscia, solo convessa		$O(1/\sqrt{\varepsilon})$	Veloce
Non-smooth, solo convessa	peggiore	$\Omega(1/\varepsilon^2)$	Molto lento

Analogia

Per raggiungere un errore $\varepsilon = 10^{-6}$:

- Nel caso **liscio e fortemente convesso**: circa $\log(10^6) \approx 14$ iterazioni.
- Nel caso **non-smooth e solo convesso**: circa $(10^6)^2 = 10^{12}$ iterazioni!!

Ecco perché non ci si aspetta che l'algoritmo A2 raggiunga la stessa precisione dell'IRLS con un numero ragionevole di iterazioni. Il gap di complessità è *abissale*.

Due altre conseguenze importanti della non-smoothness [5, slide 8–9, set 5]:

- **Il passo fisso non funziona:** a differenza del gradiente, il metodo del subgradiente con stepsize costante non converge (oscilla attorno alla soluzione senza mai raggiungerla).
- **La norma del subgradiente non è un criterio di arresto:** per funzioni lisce, $\|\nabla f(x^*)\| = 0$ al minimo. Per funzioni non-smooth, $\|g\|$ può essere grande anche nel punto ottimo (perché il subdifferenziale è un *intervallo*, non un punto).

4.3 Metodi del subgradiente: la relazione fondamentale

Intuizione

Il metodo del subgradiente ha la stessa struttura del metodo del gradiente: “fai un passo nella direzione opposta al (sub)gradiente”. Ma le sue proprietà sono radicalmente diverse. Il risultato fondamentale è la **relazione fondamentale**, che dice: anche se il valore della funzione può *peggiorare* ad un passo, la *distanza dal minimo* diminuisce (a patto che il passo sia scelto bene).

Questa è una differenza concettuale enorme rispetto ai metodi smooth: lì ogni passo migliora il valore della funzione; qui ogni passo avvicina al minimo, ma può peggiorare il valore.

Teorema 4.5 (Relazione fondamentale [5, slide 10, set 5]). Per f convessa, $g^i \in \partial f(x^i)$, e $x^{i+1} = x^i - \alpha^i g^i$:

$$\|x^{i+1} - x^*\|^2 \leq \|x^i - x^*\|^2 + \underbrace{2\alpha^i(f^* - f(x^i))}_{\leq 0 \text{ (buono!)}} + \underbrace{(\alpha^i)^2 \|g^i\|^2}_{\geq 0 \text{ (cattivo!)}}. \quad (5)$$

Dimostrazione. Espandendo il quadrato:

$$\begin{aligned} \|x^{i+1} - x^*\|^2 &= \|x^i - \alpha^i g^i - x^*\|^2 \\ &= \|x^i - x^*\|^2 - 2\alpha^i \langle g^i, x^i - x^* \rangle + (\alpha^i)^2 \|g^i\|^2. \end{aligned}$$

Dalla disuguaglianza del subgradiente: $f(x^*) \geq f(x^i) + \langle g^i, x^* - x^i \rangle$, cioè $\langle g^i, x^i - x^* \rangle \geq f(x^i) - f^*$. Sostituendo: $\leq \|x^i - x^*\|^2 - 2\alpha^i(f(x^i) - f^*) + (\alpha^i)^2 \|g^i\|^2$. $\square$

Intuizione

Come leggere la formula (5):

- Il termine $2\alpha^i(f^* - f(x^i))$ è **negativo** (perché $f^* \leq f(x^i)$): questo è il termine “buono” che ci avvicina al minimo. Più siamo lontani dall’ottimo ($f(x^i) \gg f^*$), più ci avviciniamo.
- Il termine $(\alpha^i)^2 \|g^i\|^2$ è **positivo**: questo è il “rumore” introdotto dal passo. Più il passo è grande, più rumore c’è.

Per convergere, bisogna che il termine buono domini quello cattivo. Questo richiede che il passo α^i sia “né troppo grande né troppo piccolo” e che tenda a zero abbastanza lentamente.

Come scegliere il passo (stepsize)? Le principali regole [5, slide 10–14, set 5]:

1. **Diminishing-Square Summable (DSS):** $\alpha^i = c/i$ (il passo diminuisce nel tempo). *Converge, ma è estremamente lento.* L’unico vantaggio: non richiede nessuna informazione a priori.
2. **Polyak stepsize:** $\alpha^i = (f(x^i) - f^*) / \|g^i\|^2$.

Intuizione

Questa è la scelta “perfetta”: annulla il termine positivo nella relazione fondamentale, garantendo che la distanza dal minimo diminuisca ad ogni passo. Il problema? Richiede il valore f^* ottimo, che è proprio ciò che stiamo cercando!

3. **Target level stepsize:** stima iterativamente f^* con un “target” che viene aggiornato man mano. È l’approccio usato nel nostro Algoritmo A2.

5 Algoritmo A2: Deflected Subgradient Method

5.1 Il problema del subgradiente puro e l'idea della deflection

Intuizione

Il metodo del subgradiente “puro” ha un grosso difetto: ad ogni passo usa *solo* l'informazione locale (il subgradiente nel punto corrente) e *dimentica tutto* ciò che ha fatto prima. È come un escursionista che decide dove andare guardando solo i suoi piedi, senza ricordare da dove è venuto.

Il risultato è un comportamento a “zig-zag”: il metodo oscilla avanti e indietro, perdendo tempo. Lo stesso problema affligge il metodo del gradiente per funzioni lisce, ed è stato risolto con le **direzioni coniugate**: mescolare la direzione corrente con quella precedente per creare un percorso più “dritto” verso il minimo.

L'idea dei **deflected subgradient methods** [2, 6] è la stessa: non usare il subgradiente puro g^i come direzione, ma “deviarlo” (deflect) mescolandolo con la direzione precedente d^{i-1} :

$$d^i = \gamma^i g^i + (1 - \gamma^i) d^{i-1}, \quad x^{i+1} = x^i - \alpha^i d^i, \quad (6)$$

dove $g^i \in \partial f(x^i)$, $d^0 = g^0$, e $\gamma^i \in (0, 1]$ controlla il livello di deflection.

Analogia

Pensa a una barca a vela. Il vento (il subgradiente) soffia in una direzione, ma la barca ha anche la sua *inerzia* (la direzione precedente). La rotta effettiva è un compromesso tra le due.

- $\gamma^i = 1$: segui solo il vento (subgradiente puro), ignori l'inerzia. Massima reattività, ma zig-zag.
- γ^i piccolo: segui soprattutto l'inerzia, “filtrando” il vento. Percorso più liscio, ma rischi di ignorare informazioni importanti.
- γ^i intermedio: il compromesso migliore.

5.2 Come scegliere la deflection e il passo

Intuizione

La scelta di γ^i (quanto deflettere) e α^i (quanto è lungo il passo) deve essere fatta con cura: se defletti troppo e fai passi troppo grandi, l'algoritmo diverge. Le regole di convergenza garantiscono che le due scelte siano “bilanciate”. Ci sono due filosofie: o scegli prima la deflection e poi adatti il passo, o viceversa.

Per garantire la convergenza teorica [5, slide 15, set 5], [2]:

Approccio 1: Stepsize-restricted (“scegli prima la deflection”). Si sceglie γ^i liberamente, poi il passo è vincolato a essere “abbastanza piccolo”:

$$\alpha^i = \frac{\beta^i (f(x^i) - f^*)}{\|d^i\|^2}, \quad \beta^i \leq \gamma^i.$$

Intuizione

La regola $\beta^i \leq \gamma^i$ dice: “più defletti (più γ^i è piccolo), più devi ridurre il passo”. Ha senso: la deflection introduce una direzione “vecchia” che potrebbe non essere più di discesa, quindi per sicurezza fai un passo più corto.

Approccio 2: Deflection-restricted (“scegli prima il passo”). Si sceglie lo stepsize (es. DSS), poi la deflection è vincolata:

$$\gamma^i \geq \frac{\alpha^{i-1} \|d^{i-1}\|^2}{(f(x^i) - f^*) + \alpha^{i-1} \|d^{i-1}\|^2}.$$

Intuizione

Man mano che ci avviciniamo all’ottimo ($f(x^i) \rightarrow f^*$), il numeratore e il denominatore diventano quasi uguali, quindi $\gamma^i \rightarrow 1$: vicino alla soluzione, la deflection scompare e si ricade nel subgradiente puro. Ha senso: lontano dalla soluzione la deflection aiuta a “velocizzare” il percorso, ma vicino alla soluzione serve la precisione del subgradiente locale.

Scelta ottimale della deflection: Per ogni i , si può scegliere il γ^i che minimizza la norma della direzione deflessa:

$$\gamma^i \in \arg \min \left\{ \|\gamma g^i + (1 - \gamma)d^{i-1}\|^2 : \gamma \in [0, 1] \right\}. \quad (7)$$

Intuizione

Perché minimizzare la norma della direzione? Perché nelle formule del Polyak stepsize il denominatore è $\|d^i\|^2$: più la direzione è “corta”, più il passo è lungo (e quindi più progresso si fa). Trovare il γ che rende d^i il più corto possibile è come trovare la direzione “più efficiente” mescolando il subgradiente corrente con la memoria.

Il problema (7) è un problema quadratico in una variabile vincolata a $[0, 1]$: ha una formula chiusa. Sia $\phi(\gamma) = \gamma^2 \|g\|^2 + 2\gamma(1 - \gamma) \langle g, d^- \rangle + (1 - \gamma)^2 \|d^-\|^2$. Il minimo non vincolato è:

$$\gamma^* = \frac{\|d^-\|^2 - \langle g, d^- \rangle}{\|g - d^-\|^2}.$$

Poi si proietta su $[0, 1]$: $\gamma^i = \min(1, \max(0, \gamma^*))$.

5.3 L’algoritmo completo con target level

Intuizione

Il Polyak stepsize richiede f^* , che non conosciamo. Il meccanismo di **target level** è un modo furbo per aggirare il problema: si mantiene una *stima* $f_{\text{ref}} - \delta$ di f^* (dove f_{ref} è il miglior valore trovato finora e $\delta > 0$ è una “speranza di miglioramento”) e si usa questa stima al posto di f^* .

Se la stima è buona (cioè $f_{\text{ref}} - \delta \approx f^*$), l’algoritmo fa progressi buoni. Se la stima è troppo ottimista ($f_{\text{ref}} - \delta < f^*$, cioè speriamo troppo), i passi diventano troppo piccoli e non si progredisce. In quel caso, dopo un po’ di iterazioni senza miglioramento, si riduce δ (“abbasso le aspettative”) e si riprova.

Algorithm 2 Deflected Subgradient con Target Level

Require: $f, w_0, i_{\max}, \beta \in (0, 2), \delta_0 > 0, R > 0, \rho \in (0, 1)$

```
1:  $r \leftarrow 0; \delta \leftarrow \delta_0; f_{\text{ref}} \leftarrow \bar{f} \leftarrow f(w_0); d_{-1} \leftarrow 0$ 
2: for  $i = 0, 1, \dots, i_{\max}$  do
3:   Calcola  $g \in \partial f(w_i)$  (un subgradiente qualsiasi)
4:   Calcola  $\gamma_i$  tramite (7) (o altra regola)
5:    $d_i \leftarrow \gamma_i g + (1 - \gamma_i) d_{i-1}$  (direzione deflessa)
6:    $\alpha_i \leftarrow \beta (f(w_i) - (f_{\text{ref}} - \delta)) / \|d_i\|^2$  (Polyak con target level)
7:    $w_{i+1} \leftarrow w_i - \alpha_i d_i$  (passo)
8:   if  $f(w_{i+1}) \leq f_{\text{ref}} - \delta/2$  then (miglioramento significativo)
9:      $f_{\text{ref}} \leftarrow \bar{f}; r \leftarrow 0$  (aggiorna target, azzerà contatore)
10:  else if  $r > R$  then (troppo a lungo senza miglioramenti)
11:     $\delta \leftarrow \delta \rho; r \leftarrow 0$  (riduci  $\delta$ : “abbassa le aspettative”)
12:  else
13:     $r \leftarrow r + \alpha_i \|d_i\|$  (accumula “distanza percorsa”)
14:  end if
15:   $\bar{f} \leftarrow \min(\bar{f}, f(w_{i+1}))$  (aggiorna il record)
16: end for
17: return  $w_{\text{best}}$  (il punto con il miglior  $\bar{f}$ )
```

In poche parole

L'algoritmo mantiene tre quantità chiave:

- $\bar{f}$: il **record value** — il miglior valore di f trovato finora. Questo è il vero “output” dell'algoritmo.
- $f_{\text{ref}} - \delta$: il **target level** — la stima di f^* usata per il Polyak stepsize.
- r : un contatore di “cammino percorso” senza miglioramenti, per decidere quando ridurre δ .

Osservazione 5.1 (Parametri). I parametri critici sono:

- δ_0 : stima iniziale del gap $f(w_0) - f^*$. Se troppo grande, il target è troppo basso e i passi sono enormi (instabilità); se troppo piccolo, i passi sono minuscoli e non si progredisce.
- $\rho \in (0, 1)$: tasso di riduzione di δ (tipicamente $\rho = 0.95$).
- R : soglia di “pazienza” prima di ridurre δ .
- $\beta \in (0, 2)$: modula l'ampiezza del passo. $\beta = 1$ è la scelta naturale.

5.4 Calcolo pratico del subgradiente per il nostro problema

Per il nostro problema specifico, $f(w) = \|Xw - y\|_2^2 + \lambda \|w\|_1$, un subgradiente è:

$$g = 2X^T(Xw - y) + \lambda s, \quad \text{con } s_i \in \partial|w_i|. \quad (8)$$

Intuizione

In pratica, per ogni componente:

- Se $w_i > 0$: $s_i = +1$ (unica scelta).
- Se $w_i < 0$: $s_i = -1$ (unica scelta).
- Se $w_i = 0$: s_i può essere qualsiasi valore in $[-1, +1]$. La scelta più comune è $s_i = 0$, che corrisponde all'elemento del subdifferenziale con norma minima e punta verso la “direzione di massima discesa”.

Nota: la parte “costosa” è il prodotto $X^T(Xw - y)$, che richiede due moltiplicazioni matrice-vettore: prima Xw (costo $O(mn)$), poi $X^T(\dots)$ (costo $O(mn)$). Totale: $O(mn)$ per iterazione — molto meno dei $O(n^3)$ dell'IRLS.

5.5 Convergenza: cosa aspettarsi

Teorema 5.2 (Convergenza del deflected subgradient [2]). *Sotto le condizioni di convergenza (stepsize-restricted o deflection-restricted) e assumendo f convessa e Lipschitz-continua, la sequenza dei record values $\bar{f}^i = \min_{h \leq i} f(w_h)$ converge al valore ottimo f^* . La complessità nel caso peggiore è $O(1/\varepsilon^2)$, come per il subgradiente standard.*

Intuizione

Notare che converge la sequenza dei **record values** $\bar{f}^i$, non necessariamente la sequenza delle iterate w_i . Questo perché il metodo non è monotono: $f(w_{i+1}) > f(w_i)$ può accadere. Per questo, l'algoritmo restituisce il *miglior punto trovato*, non l'ultimo.

Anche se la complessità nel caso peggiore ($O(1/\varepsilon^2)$) è identica a quella del subgradiente puro, la deflection migliora *drammaticamente* il comportamento pratico riducendo lo zig-zag e sfruttando la “memoria” delle iterazioni precedenti.

Tuttavia, non illudiamoci: per le ragioni spiegate nella Sezione 4.2, raggiungere accuratissime elevate ($\varepsilon < 10^{-4}$) con un metodo del subgradiente richiede un numero proibitivo di iterazioni.

6 Proprietà attese dalla teoria

Intuizione

Questa sezione riassume cosa ci aspettiamo di vedere quando eseguiamo gli esperimenti. Conoscere le proprietà teoriche *prima* di implementare è fondamentale: se il codice si comporta diversamente dalla teoria, c'è un bug!

6.1 Per l'algoritmo A1 (IRLS)

- **Convergenza:** $f(w_k)$ converge, ma possibilmente non in modo monotono (i valori possono oscillare leggermente, specialmente all'inizio).
- **Velocità:** convergenza tipicamente **lineare**. In un grafico con asse y logaritmico ($\log$ di $f(w_k) - f^*$) contro iterazioni, dovremmo vedere una **retta** (o quasi).
- **Accuratezza:** può raggiungere soluzioni molto accurate, limitata solo dal threshold ε_{thr} .
- **Costo:** poche iterazioni (10–50), ma costose ($O(n^3/3)$ ciascuna con Cholesky).
- **Sparsità:** la soluzione tende ad avere molte componenti a zero (o molto vicine a zero), come ci si aspetta dalla regolarizzazione L_1 .

6.2 Per l'algoritmo A2 (Deflected Subgradient)

- **Convergenza:** il record value $\bar{f}^i$ converge a f^* , ma la sequenza è **non monotona**: $f(w_{i+1}) > f(w_i)$ è possibile (e frequente!).
- **Velocità:** convergenza **sublineare**. Nel grafico logaritmico, la curva “si appiattisce” man mano (al contrario della retta di IRLS). Complessità nel caso peggiore: $O(1/\varepsilon^2)$.
- **Accuratezza:** difficile raggiungere $\varepsilon < 10^{-4}$ con un numero ragionevole di iterazioni.
- **Costo:** molte iterazioni (migliaia), ma economiche ($O(mn)$ ciascuna).
- **Criterio di arresto:** non c'è un criterio affidabile; si monitora il record value e il “budget” di iterazioni.

6.3 Chi “vince”? Confronto A1 vs A2

	A1 (IRLS)	A2 (Defl. Subgradient)
Tipo	Smooth (sotto-problema)	Non-smooth
Costo/iterazione	Alto ($O(n^3)$ o $O(kn^2)$)	Basso ($O(mn)$)
Num. iterazioni	Poche (10–50)	Molte (migliaia)
Convergenza	Tipicamente lineare	Sublineare $O(1/\varepsilon^2)$
Alta accuratezza	Sì	Molto difficile
Parametri	ε_{thr} , solver lineare	$\delta_0, \rho, R, \beta, \gamma$

In poche parole

IRLS e **deflected subgradient** sono complementari:

- IRLS fa poche iterazioni costose e raggiunge alta precisione. È come un chirurgo: lento e preciso.
- Il deflected subgradient fa tante iterazioni economiche ma fatica ad arrivare in fondo. È come un corridore: veloce all'inizio ma si stanca.

In pratica, per soluzioni a *bassa* accuratezza ($\varepsilon \sim 10^{-2}$), il deflected subgradient potrebbe essere competitivo in tempo CPU. Per soluzioni ad *alta* accuratezza ($\varepsilon < 10^{-6}$), IRLS domina nettamente.

7 Considerazioni aggiuntive

7.1 Scelta della funzione di attivazione

Intuizione

La funzione $\sigma(\cdot)$ trasforma i dati di input attraverso lo strato nascosto dell'ELM. La scelta influisce sulla matrice X e quindi sulle proprietà numeriche del problema (condizionamento, rango, ecc.).

- **Sigmoide:** $\sigma(z) = 1/(1 + e^{-z})$. Output in $(0, 1)$, quindi le colonne di X sono limitate. Possibili problemi di saturazione: se $W_1 x$ è molto grande o molto piccolo, tutti gli output diventano ≈ 0 o ≈ 1 , e le colonne di X sono quasi uguali (cattivo condizionamento).
- **ReLU:** $\sigma(z) = \max(0, z)$. Semplice e veloce, ma se i pesi W_1 sono tali che $W_1 x < 0$ per molti campioni, le colonne corrispondenti sono tutte zero (“neuroni morti”, e X perde rango).
- **Tanh:** $\sigma(z) = \tanh(z)$. Output in $(-1, 1)$, centrata in zero. Spesso un buon compromesso.

7.2 Confronto con solutori di riferimento

Intuizione

Il testo del progetto dice “no off-the-shelf solvers allowed” per risolvere il LASSO. Ma confrontare i risultati con un solutore esterno è **essenziale per la validazione**: se il nostro algoritmo e `sklearn.linear_model.Lasso` danno la stessa soluzione (o quasi), possiamo essere ragionevolmente sicuri che il codice è corretto.

In particolare, il solutore esterno fornisce un valore di riferimento f_{ref}^* che serve per calcolare il gap $f(w_k) - f^*$ nei grafici di convergenza. Senza questo riferimento, non potremmo tracciare i grafici!

7.3 Smoothness e strong convexity del nostro problema

Definizione 7.1 (L -smoothness [5, slide 4, set 4]). $f \in C^1$ è L -smooth se $\|\nabla f(x) - \nabla f(z)\| \leq L\|x - z\|$ per ogni x, z . Informalmente: il gradiente “non cambia troppo velocemente”.

Definizione 7.2 (τ -convessità forte [5, slide 4, set 4]). f è τ -fortemente convessa se $f(z) \geq f(x) + \langle \nabla f(x), z - x \rangle + \frac{\tau}{2} \|z - x\|^2$. Informalmente: la ciotola non è troppo “piatta” — ha una curvatura minima garantita.

Per la *parte quadratica* del nostro problema, $g(w) = \|Xw - y\|_2^2$:

- $L = 2\lambda_{\max}(X^T X)$ — la smoothness è determinata dal più grande autovalore di $X^T X$.
- $\tau = 2\lambda_{\min}(X^T X)$ — la strong convexity è determinata dal più piccolo autovalore. Se X non ha rango pieno, $\tau = 0$ e g non è fortemente convessa.
- Il numero di condizione è $\kappa = L/\tau = \kappa(X)^2$.

Attenzione

La funzione *totale* $f = g + h$ non è smooth (a causa del termine L_1), anche se la parte g lo è. Questo è il motivo fondamentale per cui servono metodi non-smooth per risolvere il problema direttamente (come il deflected subgradient), oppure trucchi per “aggirare” la non-smoothness (come l’IRLS).

8 Mappa delle lezioni: cosa studiare e dove trovarlo

Intuizione

Questa sezione è una guida allo studio: per ogni argomento necessario al progetto, indica **esattamente** in quali file PDF delle lezioni trovarlo, con le pagine/slide rilevanti. Le lezioni sono divise in due gruppi: **Ottimizzazione** (set 0–7, prof. Frangioni) e **Algebra Lineare Numerica** (lectures 0–21, prof. Cortinovis). Non serve studiare tutto: i file sono classificati come **essenziali**, **utili** o **non necessari**.

8.1 Lezioni di Ottimizzazione

ESSENZIALE 5-nonsmooth unconstrained optimization.pdf (98 pagine)

Questo è il file **più importante per il progetto**. Contiene la teoria di base dell'algoritmo A2 e la motivazione del problema LASSO.

- **Slide 1–2: Motivazione I — Stochastic Gradient.** Contesto ML: perché servono metodi che funzionino con informazione di primo ordine imprecisa. Background utile, non strettamente necessario.
- **Slide 3: Motivazione II — Regularizzazione non-differenziabile.** *Fondamentale.* Introduce la regolarizzazione LASSO $\|w\|_1$ come alternativa convessa alla norma L_0 . Spiega perché la L_1 promuove sparsità e perché è non-differenziabile. È la motivazione diretta del nostro problema.
- **Slide 4–5: I metodi smooth falliscono.** Esempio concreto (LASSO con $\mu = 10$) che mostra come il gradiente standard si blocca nei punti di non-differenziabilità. Spiega *perché* servono metodi diversi.
- **Slide 5–7: Subgradienti e subdifferenziale.** *Fondamentale.* Definizione di subgradiente, subdifferenziale $\partial f(x)$, proprietà (insieme convesso e compatto, regola della somma), calcolo per la norma L_1 . Condizione di ottimalità $0 \in \partial f(x^*)$.
- **Slide 8: Complessità dell'ottimizzazione non-smooth.** *Fondamentale.* Tabella comparativa smooth vs. non-smooth: mostra il salto drammatico di complessità ($O(\log 1/\varepsilon)$ vs. $O(1/\varepsilon^2)$). Da citare nel report per giustificare le prestazioni attese dell'algoritmo A2.
- **Slide 9: Passo fisso e criterio di arresto.** Il passo fisso “non funziona” per l'ottimizzazione non-smooth. La norma del subgradiente non è un criterio di arresto valido ($\|g\|$ può essere grande anche in x^*).
- **Slide 10: Relazione fondamentale e stepsize DSS.** *Fondamentale.* La disuguaglianza $\|x^{i+1} - x^*\|^2 \leq \|x^i - x^*\|^2 + 2\alpha^i(f^* - f^i) + (\alpha^i)^2 \|g^i\|^2$ e la regola DSS ($\alpha^i = c/i$).
- **Slide 12–13: Polyak stepsize.** *Fondamentale.* Formula $\alpha^i = (f^i - f^*) / \|g^i\|^2$, proprietà (ogni passo riduce la distanza da x^*), complessità $O(1/\varepsilon^2)$. Record value f^i .
- **Slide 14: Target level stepsize.** *Fondamentale.* Meccanismo per stimare f^* iterativamente: f_{ref} , soglia δ , regole di aggiornamento. Direttamente usato nell'Algoritmo A2.
- **Slide 15: Deflected subgradient.** *Fondamentale.* Formula $d^i = \gamma^i g^i + (1 - \gamma^i) d^{i-1}$, le due regole di convergenza (stepsize-restricted e deflection-restricted), formula chiusa per γ^i . Riferimento a d'Antonio–Frangioni 2009. È il cuore dell'algoritmo A2.

- **Slide 16–18: Smoothed gradient methods.** Approccio alternativo (non richiesto dal progetto). Utile per capire il confronto nella slide di wrap-up.
- **Slide 19–21: Bundle methods.** Non necessari per il progetto, ma utili per contesto: rappresentano l'approccio “sofisticato” all'ottimizzazione non-smooth.
- **Slide 22 (wrap-up): Confronto finale.** “Forget high accuracy unless you fight hard”: conferma che i metodi del subgradiente non possono raggiungere accuratèzze elevate.

ESSENZIALE 4-smooth unconstrained optimization.pdf (98 pagine)

Contiene la teoria dei metodi di discesa smooth, necessaria per capire sia i sotto-problemi dell'IRLS sia il concetto di “deflection”.

- **Slide 1–3: Metodo del gradiente.** Struttura base: $x \leftarrow x - \alpha \nabla f(x)$. Convergenza con exact line search.
- **Slide 4–8: Line search inesatta (Armijo, Wolfe).** Non direttamente usato nel progetto (l'IRLS risolve sotto-problemi esatti), ma utile come background.
- **Slide 9–10: L-smoothness e convessità forte.** *Fondamentale.* Definizioni di L -smoothness ($\|\nabla f(x) - \nabla f(z)\| \leq L\|x - z\|$), τ -convessità forte, e numero di condizione $\kappa = L/\tau$. Servono per analizzare la convergenza di entrambi gli algoritmi.
- **Slide 11–14: Metodo del gradiente con passo fisso.** Convergenza con $\alpha = 1/L$: $O(1/\varepsilon)$ per funzioni convesse, $O(\log 1/\varepsilon)$ per funzioni fortemente convesse. Da confrontare con la complessità dei metodi non-smooth.
- **Slide 15–16: Metodi twisted (Newton, quasi-Newton).** Background: come l'Hessiano migliora la convergenza. Non direttamente necessario.
- **Slide 17–19: Deflected gradient methods (NCG, Heavy Ball, Accelerated Gradient).** *Molto utile.* Il concetto di *deflection* nel caso smooth: $d^i = -\nabla f(x^i) + \beta^i d^{i-1}$. I metodi NCG (4 formule per β : Fletcher-Reeves, Polak-Ribière, Hestenes-Stiefel, Dai-Yuan) e Heavy Ball (convergenza non-monotona, tasso $r = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$). Questi sono gli *analoghi smooth* del deflected subgradient — capirli aiuta enormemente a capire A2.

UTILE 3-unconstrained optimality.pdf (76 pagine)

Prerequisiti teorici: ottimalità e convessità.

- **Slide 1–3: Norme.** Definizione di p -norme, inclusa la norma L_1 (Lasso) e L_0 . Le palle unitarie in $\mathbb{R}^2$. Utile per capire la geometria della regolarizzazione.
- **Slide 4–8: Gradiente, Hessiano, Jacobiano.** Definizioni fondamentali. Gradiente come direzione di massima crescita, Hessiano come matrice di curvatura.
- **Slide 9–12: Condizioni di ottimalità.** Condizione necessaria $\nabla f(x^*) = 0$, condizione sufficiente $\nabla^2 f(x^*) \succ 0$. Caso convesso: $\nabla f(x^*) = 0 \Leftrightarrow$ minimo globale.
- **Slide 13–16: Funzioni convesse.** Definizione, proprietà (somma di convesse è convessa, livelli convessi), matrici semi-definite positive. *Importante* per giustificare che il nostro problema ha un unico minimo.

NON NECESSARIO**Altri set di ottimizzazione**

- **0-introduction.pdf (56 pagine):** Introduzione al corso, motivazioni, tipologie di problemi. Solo background generale — da sfogliare, non da studiare.
- **1-simple problems.pdf (310 pagine):** Problemi semplici, funzioni lineari/quadratiche, metodo del gradiente per quadratiche. Molto lungo; le parti rilevanti (quadratiche, gradiente) sono coperte meglio nel set 4.
- **2-univariate optimization.pdf (94 pagine):** Ottimizzazione univariata (line search, golden section, ...). Non direttamente necessario per il progetto.
- **6-constrained optimality and duality.pdf (201 pagine):** Ottimizzazione vincolata, KKT, dualità lagrangiana. Non rilevante: il nostro problema è *non vincolato*.
- **7-constrained optimization.pdf (75 pagine):** Algoritmi per ottimizzazione vincolata (active set, projected gradient, barrier). Non rilevante.

8.2 Lezioni di Algebra Lineare Numerica**ESSENZIALE****4-leastsquares-normal.pdf (10 pagine)**

Il file *NLA* più importante per il progetto. Copre le equazioni normali $X^T X w = X^T y$, che sono il cuore del sotto-problema IRLS.

- Condizione di rango pieno per l'unicità della soluzione.
- Pseudoinversa $X^+ = (X^T X)^{-1} X^T$.
- Collegamento con la proiezione ortogonale.
- Costo computazionale: $O(mn^2 + n^3/3)$.

ESSENZIALE**20-cho1.pdf (10 pagine)**

Fattorizzazione di Cholesky $Q = LL^T$ per matrici simmetriche definite positive. *Direttamente usata* nell'IRLS per risolvere il sistema $(X^T X + 2\lambda W_k^T W_k)w = X^T y$.

- Derivazione dalla fattorizzazione LU sfruttando la simmetria.
- $Q = LDL^T$ e poi $Q = LL^T$ con $L \leftarrow L\sqrt{D}$.
- Costo: $n^3/3$ (metà di LU).
- Esistenza garantita per $Q \succ 0$.

ESSENZIALE**5-CG.pdf (34 pagine)**

Il Gradiente Coniugato: metodo iterativo alternativo per risolvere i sotto-problemi IRLS, specialmente per n grande.

- **Pagine 1–4:** Motivazione (matrici sparse e grandi), formulazione come problema di ottimizzazione $\min \frac{1}{2} x^T Q x - v^T x$.
- **Pagine 5–12:** Direzioni Q -coniugate, costruzione iterativa, implementazione dell'algoritmo CG.
- **Pagine 13–18:** Spazi di Krylov $\mathcal{K}_n(Q, v) = \text{span}(v, Qv, \dots, Q^{n-1}v)$. Il CG come “best approximation in Krylov space”.

- **Pagine 19–26:** *Convergenza*. Dipendenza dalla distribuzione degli autovalori di Q . Bound $\leq 2((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1))^k$. Convergenza in cluster di autovalori. Convergenza lineare con rate dipendente da $\kappa(Q)$.
- **Pagine 27–34:** Esempi pratici, `pcg` in Matlab.

ESSENZIALE 3-intro-leastsquares.pdf (13 pagine)

Introduzione ai problemi dei minimi quadrati. Spiega *cosa* stiamo risolvendo.

- Il problema $\min \|Ax - y\|$ come “trovare la combinazione lineare di colonne di A più vicina a y ”.
- Solvibilità: quando $n < m$ (sistema sovradeterminato), la soluzione esatta in generale non esiste.
- Proiezione ortogonale: Ax^* è la proiezione di y sullo spazio colonna di A .
- Residuo $r = y - Ax^*$ ortogonale allo spazio colonna: $A^T r = 0$.

UTILE 12-conditioning.pdf (13 pagine)

Il numero di condizione e il confronto tra algoritmi per i minimi quadrati.

- Confronto di costo: equazioni normali ($\frac{4}{3}n^3$ se $m \approx n$, mn^2 se $m \gg n$) vs. QR ($2mn^2$) vs. SVD ($\approx 13n^3$).
- *Fondamentale*: esempio numerico che mostra come le equazioni normali possono dare risultati imprecisi quando $\kappa(A)$ è grande, mentre QR e SVD sono più stabili.
- Motivo: $\kappa(A^T A) = \kappa(A)^2$, quindi le equazioni normali “peggiorano il condizionamento al quadrato”.

UTILE 13-conditioning-least-squares.pdf (10 pagine)

Condizionamento specifico del problema dei minimi quadrati.

- Definizione formale di $\kappa(A) = \|A\| \|A^{-1}\| = \sigma_{\max}/\sigma_{\min}$.
- Bound sull'errore relativo: $\|\tilde{x} - x\| / \|x\| \leq \kappa(A) \cdot \|\tilde{y} - y\| / \|y\|$.
- Utile per capire *quando* l'IRLS potrebbe dare risultati imprecisi.

UTILE 19-lu.pdf (22 pagine)

Fattorizzazione LU (eliminazione di Gauss). Background per capire la fattorizzazione di Cholesky (lecture 20).

- $A = LU$: fattorizzazione in matrice triangolare inferiore $\times$ superiore.
- Pivoting per la stabilità numerica.
- Costo: $\frac{2}{3}n^3$ (il doppio di Cholesky, che sfrutta la simmetria).

UTILE 21-largescale-examples.pdf (12 pagine)

Esempi pratici di risoluzione di sistemi lineari di grandi dimensioni.

- Confronto tra Cholesky denso, Cholesky sparso (con riordinamento `symrcm`), e CG (`pcg`).
- Mostra che CG può essere ordini di grandezza più veloce di Cholesky per matrici sparse grandi.
- Rilevante per decidere quale solver usare nell'IRLS al variare di n .

UTILE 2-orthogonality.pdf (14 pagine)

Norme vettoriali, matrici ortogonali. Background.

- Definizione di norma 2 e norma p .
- Matrici ortogonali: $U^T U = I$, preservano la norma 2.
- Proiezioni ortogonali.

UTILE 7-matrixnorm.pdf (6 pagine)

Norme matriciali indotte, norma di Frobenius.

- Definizione di $\|A\|_2 = \max_{\|v\|=1} \|Av\|$.
- $\|A\|_2 = \sigma_{\max}(A)$ (il più grande valore singolare).
- Proprietà: sub-moltiplicatività $\|AB\| \leq \|A\| \|B\|$.

NON NECESSARIO Lectures NLA non rilevanti per il progetto

- 0-intro.pdf (9 pag.): Informazioni sul corso. Solo introduttivo.
- 1-linalg.pdf (18 pag.): Ripasso di algebra lineare (prodotti matrice-vettore, combinazioni lineari). Prerequisito.
- 6-SVD.pdf (8 pag.), 8-lab-SVD.pdf (22 pag.), 11-leastsquares-SVD.pdf (18 pag.): SVD e applicazioni. Non usata direttamente nel progetto (usiamo Cholesky/CG, non SVD).
- 9-QR.pdf (16 pag.), 10-leastsquares-QR.pdf (6 pag.): Fattorizzazione QR. Alternativa alle equazioni normali ma non usata nel nostro progetto.
- 14-stability.pdf (13 pag.), 15-stability-and-residual.pdf (10 pag.), 16-stability-least-squares.pdf (8 pag.): Stabilità degli algoritmi, analisi del residuo. Utile per capire se il codice è corretto, ma non necessario per l'implementazione.
- 17-arnoldi.pdf (11 pag.), 18-GMRES.pdf (11 pag.): Metodi di Krylov per sistemi non-SPD (Arnoldi, GMRES). Non rilevanti: il nostro sistema è sempre SPD.

8.3 Tabella riassuntiva: priorità di studio

Prior.	File	Pag.	Argomento per il progetto
ESSENZIALI — studiare a fondo			
1	OPT/5-nonsmooth.pdf	98	Subgradiente, deflection, target level
2	OPT/4-smooth.pdf	98	Smooth methods, deflected gradient, L -smooth
3	NLA/4-leastsquares-normal.pdf	10	Equazioni normali (cuore IRLS)
4	NLA/20-chol.pdf	10	Cholesky (solver per IRLS)
5	NLA/5-CG.pdf	34	Gradiente Coniugato (solver alternativo)
6	NLA/3-intro-leastsquares.pdf	13	Cosa sono i minimi quadrati
UTILI — leggere per contesto			
7	OPT/3-unconstrained.pdf	76	Norme, convessità, condizioni di ottimalità
8	NLA/12-conditioning.pdf	13	Condizionamento, confronto metodi
9	NLA/13-conditioning-LS.pdf	10	$\kappa(A)$, errore relativo
10	NLA/19-lu.pdf	22	LU (background per Cholesky)
11	NLA/21-largescale.pdf	12	Cholesky vs CG in pratica
NON NECESSARI — saltare			
–	OPT/0,1,2,6,7	736	Intro, univariata, vincolata
–	NLA/0,1,6,8,9,10,11,14-18	174	SVD, QR, GMRES, stabilità

In poche parole

Percorso di studio consigliato:

1. Inizia da OPT/3 (slide 1–16): norme, gradiente, Hessiano, convessità.
 2. Poi NLA/3 e NLA/4: minimi quadrati ed equazioni normali.
 3. Poi NLA/20: Cholesky. Ora capisci il sotto-problema dell'IRLS.
 4. Poi OPT/4 (slide 9–19): L -smoothness, convessità forte, deflected gradient.
 5. Poi NLA/5: Gradiente Coniugato. Ora capisci i solver alternativi.
 6. Infine OPT/5 (tutte): subgradienti, Polyak, target level, deflected subgradient.
 7. Opzionale: NLA/12–13 per il condizionamento, NLA/21 per gli esempi pratici.
- Totale: circa **280 pagine** (di slides, non di testo denso), di cui le essenziali sono circa **175**.

Riferimenti bibliografici

- [1] S. Bubeck, *Convex Optimization: Algorithms and Complexity*, Foundations and Trends in Machine Learning 8(3–4), 231–357, 2015. arXiv:1405.4980v2.
- [2] G. d'Antonio, A. Frangioni, *Convergence Analysis of Deflected Conditional Approximate Subgradient Methods*, SIAM Journal on Optimization 20(1), 357–386, 2009.
- [3] A. Frangioni, *Standard Bundle Methods: Untrusted Models and Duality*, in: Numerical Nonsmooth Optimization, Springer, 2020.
- [4] J. Nocedal, S.J. Wright, *Numerical Optimization*, 2nd edition, Springer, 2006.

- [5] A. Frangioni, Slides del corso *Computational Mathematics for Learning and Data Analysis*, sezione Ottimizzazione, A.A. 2025/26. In particolare: set 0 (Introduction), set 3 (Unconstrained optimality), set 5 (Nonsmooth unconstrained optimization).
- [6] A. Frangioni, *Nonsmooth unconstrained optimization* (slide set 5), Slides del corso CM, A.A. 2025/26.
- [7] F. Poloni, Slides del corso *Computational Mathematics for Learning and Data Analysis*, sezione Algebra Lineare Numerica, A.A. 2025/26. In particolare: lectures 3–4 (Least squares, Normal equations), lecture 5 (CG), lecture 20 (Cholesky).
- [8] A. Frangioni, *Optimization and Learning — Notes 25/26*, dispense del corso. Cap. 5: Nonsmooth Convex Unconstrained Optimization (§5.1.2: Lasso regularisation, §5.4: Subgradient methods). Cap. 4: Smooth optimization (§4.3: Deflected gradient methods).
- [9] L.N. Trefethen, D. Bau, *Numerical Linear Algebra*, SIAM, 1997.